NEMOX AI RICO-7

VOLUME III | OPERATIONS & VERIFICATION | SYNTX FORM

Build Developer Tools to Better Understand Users · nemox AI, Munich

SYNTX FIELD-BASED SYSTEM ARCHITECTURE | SOURCES: QUICK_START.MD · TEST_REPORT.MD · VOLUMES I AND II | 1 NOVEMBER 2026

- Vol. I · Complete Technical Architecture
- Vol. II · Module & Developer Guide
- Vol. III · Operations & Verification

CONFIDENTIAL | PROPRIETARY | NO USE WITHOUT WRITTEN APPROVAL

This document and the operations, verification and SYNTX representation it contains are the confidential work of Ottavio Braun, SYNTX SYSTEM. No right of use is granted. Any use, reproduction, disclosure, adaptation, implementation or AI training — in whole or in part — requires the prior, explicit, written approval of Ottavio Braun for a named purpose. Full terms: final page.

NUTZUNG, WEITERGABE ODER UMSETZUNG – GANZ ODER TEILWEISE – NUR NACH AUSDRÜCKLICHER SCHRIFTLICHER FREIGABE DURCH OTTAVIO BRAUN.

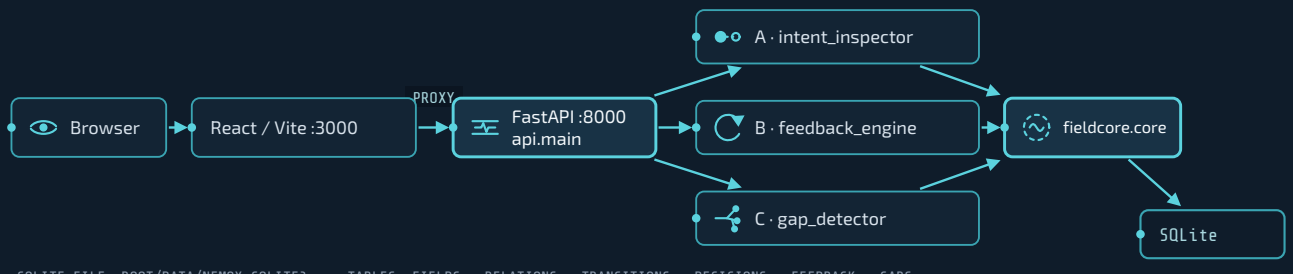
STATUS SEMIOTICS ● EXECUTED / VERIFIED ◐ INDIRECTLY EXERCISED ◌ CONFIGURATION ◆ NOT EXECUTED / PENDING

01 SYSTEM OVERVIEW

SHARED PLATFORM · THREE DEVELOPER TOOLS

◆ One shared platform — React / Vite frontend, FastAPI backend, shared field core on SQLite — carries three deterministic developer tools for understanding user requests, decisions and gaps.

S1 SYSTEM ARCHITECTURE



SQLITE FILE: ROOT/DATA/NEMOX.SQLITE3 · TABLES: FIELDS · RELATIONS · TRANSITIONS · DECISIONS · FEEDBACK · GAPS

A User Intent & Context Inspector

Interprets a freight request with a historical reference; extracts constraints, candidates and unresolved references.

B Recursive Feedback & Decision Trace Engine

Records a decision, a correction and its confirmation; reuses confirmed corrections in subsequent decisions.

C User Understanding Gap Detector

Finds capability-required attributes that are missing and asks for the blocking frontier until complete.

© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

02 INSTALLATION & QUICK START

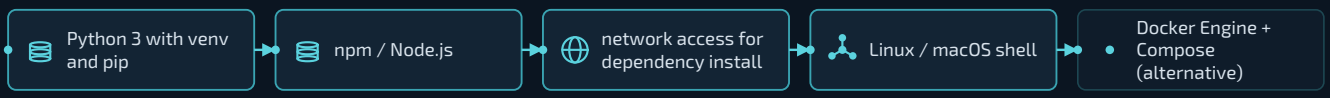
QUICK_START.MD · COMPLETE

- The quick start as recorded in QUICK_START.md, expanded with the operating detail documented in Volumes I and II.

QUICK_START.MD · VERBATIM

```
# QUICK_START.md
› Extract 01_SOFTWARE/NEMOX_AI_RICO7_COMPLETE_SOFTWARE.zip.
› cd nemox-developer-tools && ./start.sh
# Browser: http://localhost:3000
# Requires Python 3, npm, network access for dependency install.
# Docker alternative:
› docker compose up --build
```

PREREQUISITES



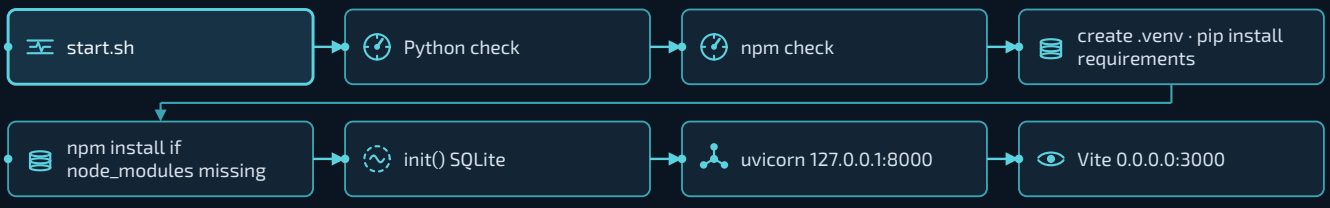
DIRECTORY STRUCTURE

PACKAGE NEMOX-DEVELOPER-TOOLS · PER VOL. II INVENTORY

```
nemox-developer-tools/
  README.md
  start.sh
  docker-compose.yml
  backend/
    api/main.py
    fieldcore/core.py
    intent_inspector/service.py
    feedback_engine/service.py
    gap_detector/service.py
    tests/test_system.py
    requirements.txt · Dockerfile
  frontend/
    src/main.tsx · src/services/api.ts · src/components/FieldGraph.tsx
    src/style.css · index.html · package.json · tsconfig.json · vite.config.ts · Dockerfile
  legacy/02_Executable_Source/NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/
    01_user_intent_inspector/ · 02_recursive_feedback_engine/
    03_understanding_gap_detector/ · docker-compose.yml
  data/nemox.sqlite3  (created at runtime by init())
```

INSTALLATION & STARTUP

./START.SH · PER VOL. I §10



LOCAL START · VOL. II §1

```
› unzip NEMOX_AI_RICO7_COMPLETE_SOFTWARE.zip
› cd nemox-developer-tools
› ./start.sh
# Browser: http://localhost:3000
# Backend health: http://localhost:8000/api/health
```

CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

APPLICATION ACCESS

ENDPOINT	ADDRESS	PURPOSE
Frontend	http://localhost:3000	three tool tabs, JSON output, FieldGraph
Backend health	http://localhost:8000/api/health	returns {ok:true}
API base	/api (Vite proxy › backend :8000)	eleven operations, §3

DOCKER

DOCKER ALTERNATIVE

```
› docker compose up --build
```

ELEMENT	CONFIGURATION
Backend container	python:3.12-slim at 8000
Frontend container	node:22-alpine at 3000
Vite proxy	DOCKER_ENV=1 targets backend:8000; local proxy targets 127.0.0.1:8000
SQLite data	bind mount ./data:/app/data

◆ DOCKER STATUS

Docker startup is not executed in the recorded environment — Docker unavailable (TEST_REPORT.md).

TROUBLESHOOTING · DOCUMENTED CONDITIONS ONLY

CONDITION	DOCUMENTED CAUSE / ACTION	SOURCE
◆ Dependencies not installed	First start requires network access to install pip / npm packages; the ZIP contains source, no node_modules and no Python virtualenv.	start.sh · requirements.txt · package.json
◆ Frontend build fails / absent	node_modules absent in the recorded environment. Run: cd frontend && npm install --no-audit --no-fund && npm run build.	frontend/package.json
◆ HTTP 422	Pydantic validation failure on a request body.	backend/api/main.py
◆ HTTP 400	ValueError — missing field, scope mismatch, invalid transition or version conflict.	backend/fieldcore/core.py
◆ HTTP 500	Unhandled sqlite3 or other exception.	backend/api/main.py
◆ Data disappeared after Reset	POST /api/reset clears every row from six tables across all scopes; global, not scoped. SQLite file remains on disk.	FieldRepository.reset()

No dedicated troubleshooting guide exists in the source files; the conditions above are those documented in Volumes I and II.

03 API REFERENCE

11 OPERATIONS · EXTRACTED FROM VOL. II §3

◆ All eleven operations with method, path, implementation, request and response contract, and the documented curl example.

Examples follow the documented request contracts. IDs are generated at runtime: the Solution B and C sequences capture them in shell variables (DECISION_ID, FEEDBACK_ID, GAP_ID) from the documented id response field, using python3 from the prerequisites.

GET `/api/health` — backend/api/main.py → health()

REQUEST none

RESPONSE {ok:true}

POST `/api/intent` — backend/api/main.py → inspect()

REQUEST {text:str,scope?:str,values?:object}

RESPONSE request,requirements,candidates,unresolved,field_ids,trace

```
› curl -s -X POST http://localhost:8000/api/intent \
    -H "Content-Type: application/json" \
    -d '{"text":"Same shipment as last week, but two days earlier."}'
```

CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

POST /api/decisions — backend/api/main.py → decide()

REQUEST {scope?:str,action:str,context?:object}

RESPONSE id,scope,action,context

```
# Solution B · step 1-2: create a decision and keep its id
› DECISION_ID=$(curl -s -X POST http://localhost:8000/api/decisions \
    -H "Content-Type: application/json" \
    -d '{"scope":"demo","action":"Carrier A","context":{"operation":"quote","route":"MUC-HAM"}}' \
    | python3 -c 'import sys,json; print(json.load(sys.stdin)["id"])')
› echo "$DECISION_ID"
```

POST /api/feedback — backend/api/main.py → correct()

REQUEST {scope?:str,decision_id:str,correction:object}

RESPONSE id,decision_id,scope,correction,status

```
# Solution B · step 3-4: correct that decision and keep the feedback id (status PROPOSED)
› FEEDBACK_ID=$(curl -s -X POST http://localhost:8000/api/feedback \
    -H "Content-Type: application/json" \
    -d '{"scope":"demo","decision_id":"'"$DECISION_ID"'",
         "correction":{"operation":"quote","route":"MUC-HAM","preferred_action":"Carrier B"}}' \
    | python3 -c 'import sys,json; print(json.load(sys.stdin)["id"])')
› echo "$FEEDBACK_ID"
```

POST /api/feedback/{fid}/confirm — backend/api/main.py → confirm()

REQUEST {scope?:str,accept:bool}

RESPONSE id,status,field

```
# Solution B · step 5: confirm the feedback returned above
› curl -s -X POST "http://localhost:8000/api/feedback/$FEEDBACK_ID/confirm" \
    -H "Content-Type: application/json" \
    -d '{"scope":"demo","accept":true}'
```

POST /api/subsequent — backend/api/main.py → subsequent()

REQUEST {scope?:str,operation:str,route:str,base_action:str}

RESPONSE baseline,result,retrieved,changed

```
# Solution B · step 6: subsequent decision for the same scope, operation and route
› curl -s -X POST http://localhost:8000/api/subsequent \
    -H "Content-Type: application/json" \
    -d '{"scope":"demo","operation":"quote","route":"MUC-HAM","base_action":"Carrier A"}'
```

GET /api/history/{scope} — backend/api/main.py → history()

REQUEST scope path

RESPONSE decisions,feedback

POST /api/gaps — backend/api/main.py → analyze()

REQUEST {text:str,scope?:str,values?:object}

RESPONSE id,capability,values,missing,blocking_frontier,questions,complete,edges

```
# Solution C · step 1-2: analyze the request and keep the gap id
› GAP_ID=$(curl -s -X POST http://localhost:8000/api/gaps \
    -H "Content-Type: application/json" \
    -d '{"scope":"demo","text":"Ship this as usual, but use the cheaper option."}' \
    | python3 -c 'import sys,json; print(json.load(sys.stdin)["id"])')
› echo "$GAP_ID"
```

CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

POST `/api/gaps/{gid}/resolve` — backend/api/main.py → resolve()

REQUEST {scope?:str,answers:object}
RESPONSE gap evaluation object

```
# Solution C · step 3: answer the blocking frontier (shipment_id), then the next frontier
› curl -s -X POST "http://localhost:8000/api/gaps/$GAP_ID/resolve" \
    -H "Content-Type: application/json" \
    -d '{"scope":"demo","answers":{"shipment_id":"SHP-100"}}'

› curl -s -X POST "http://localhost:8000/api/gaps/$GAP_ID/resolve" \
    -H "Content-Type: application/json" \
    -d '{"scope":"demo","answers":{"rate_options":"A,B","delivery_deadline":"2026-11-10"}}'
```

GET `/api/graph/{scope}` — backend/api/main.py → FieldRepository.graph()

REQUEST scope path
RESPONSE fields,relations,transitions

POST `/api/reset` — backend/api/main.py → FieldRepository.reset()

REQUEST {}
RESPONSE {reset:true}

```
› curl -s -X POST http://localhost:8000/api/reset \
    -H "Content-Type: application/json" \
    -d '{}'
```

◆ CONTRACT BOUNDARY · VOL. I §7

FastAPI returns 422 for Pydantic validation failures and 400 for ValueError; unhandled sqlite3 or other exceptions can return 500. No authentication, authorization, rate limiting or cross-tenant enforcement on reset. GET history and graph are scope-filtered by URL.

◆ EXAMPLE STATUS · NOT EXECUTED

The corrected examples have not been executed in the recorded environment. Not documented in the source contracts and therefore to be checked against backend/feedback_engine/service.py: the key names inside context and correction (operation, route, preferred_action, taken from the documented retrieval logic) and the example base action Carrier A. Answer keys for /resolve are the documented capability attributes shipment_id, rate_options and delivery_deadline.

04 THREE DEVELOPER TOOL WORKFLOWS

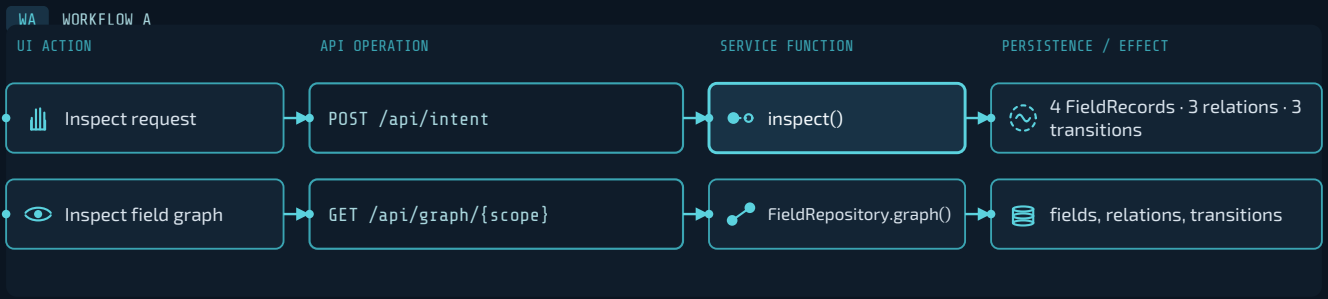
INPUT · PROCESSING · API · OUTPUT

◆ Each tool as an operational workflow: the documented input, the UI actions, the API operations and service functions they call, what is persisted, and the expected output.

A Solution A · User Intent & Context Inspector

INTENT INSPECTOR TAB

Input: default text Same shipment as last week, but two days earlier.



Output: delivery_delta_days=-2, two historical candidates and unresolved historical_shipment_reference; field graph and transitions.

Processing: regex extraction › HISTORY candidates › create 4 fields › 3 relations › resolver trace.

CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

B Solution B · Recursive Feedback & Decision Trace Engine

FEEDBACK ENGINE TAB

Input: carrier decision, correction quote / MUC-HAM, confirmation, subsequent operation and route.

WB WORKFLOW B

UI ACTION	API OPERATION	SERVICE FUNCTION	PERSISTENCE / EFFECT
1 Record decision	POST /api/decisions	decide()	decisions row
2 Submit correction	POST /api/feedback	correct()	feedback PROPOSED
3 Confirm correction	POST /api/feedback/{fid}/confirm	confirm()	status CONFIRMED + Feedback FieldRecord
4 Subsequent decision	POST /api/subsequent	subsequent() · retrieve()	new decisions row
History	GET /api/history/{scope}	history()	decisions, feedback

Output: the confirmed quote/MUC-HAM correction changes the chosen action to Carrier B. The original decision remains in the decisions table.

Retrieval filter: confirmed feedback › scope match › operation match › route match › last correction › action override.

C Solution C · User Understanding Gap Detector

GAP DETECTOR TAB

Input: instruction text, then clarification answers SHP-100 · A,B · 2026-11-10.

WC WORKFLOW C

UI ACTION	API OPERATION	SERVICE FUNCTION	PERSISTENCE / EFFECT
Analyze gaps	POST /api/gaps	analyze() · evaluate()	Request FieldRecord + gaps row › frontier shipment_id
Enter SHP-100, submit	POST /api/gaps/{gid}/resolve	resolve() · evaluate()	values_json › frontier rate_options, delivery_deadline
Enter A,B and 2026-11-10, submit	POST /api/gaps/{gid}/resolve	resolve() · evaluate()	values_json › complete=true

Output: complete=true once shipment_id, rate_options and delivery_deadline are present.

Dependency graph: shipment_id › rate_options; shipment_id › delivery_deadline. Frontier = nodes whose direct dependencies are present.

◆ RESET · GLOBAL DESTRUCTIVE

Reset all calls POST /api/reset and clears every row from six tables, across all scopes. It is a global destructive demo reset, not scoped. Existing SQLite file remains on disk.

CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

05 TEST EXECUTION

TEST_REPORT.MD · COMPLETE

- The recorded test report, its commands, the five backend test cases and the paths that remain unverified. No results beyond those recorded are stated.

TEST_REPORT.MD · VERBATIM

```
# TEST_REPORT.md
› Backend: PYTHONPATH=backend python -m pytest -q backend/tests/test_system.py -> 5 passed (2026-11-01).
› Frontend build: not executed, node_modules absent.
› Docker startup: not executed, Docker unavailable.
› Browser E2E: not executed.
› ZIP CRC integrity: checked after packaging.
```

RECORDED EXECUTION RESULTS

CHECK	COMMAND / SCOPE	RECORDED RESULT
● Backend tests	`PYTHONPATH=backend python -m pytest -q backend/tests/test_system.py`	5 passed · 2026-11-01
● ZIP CRC integrity	`package archive`	checked after packaging
◆ Frontend build	`cd frontend && npm install --no-audit --no-fund && npm run build`	not executed · node_modules absent
◆ Docker startup	`docker compose up --build`	not executed · Docker unavailable
◆ Browser E2E	–	not executed

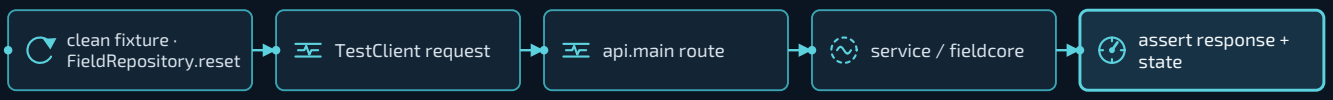
Failed tests: none recorded.

BACKEND TEST CASES · BACKEND/TESTS/TEST_SYSTEM.PY

FASTAPI TESTCLIENT · FIELDREPOSITORY.RESET FIXTURE (CLEAN)

TEST	COVERS	FIELD / FUNCTION	MODULE
● `test_intent_trace`	Intent trace — relative delivery constraint and historical context candidates	`F02 / F04 / F05 / F03 · inspect()`	A
● `test_feedback_scope_and_confirm`	Feedback confirmation and route-scoped reuse — decision correlation, confirm and reuse	`F06 / F07 / F12 · correct(), confirm(), retrieve(), subsequent()`	B
● `test_gap_resolution`	Gap clarification — dependency frontier and clarification update	`F10 · capability(), evaluate(), resolve()`	C
● `test_state_validation`	Transition validation — transition guard and version	`F09 · FieldRepository.transition()`	core
● `test_relation_scope`	Relation scope mismatch — relation source / target scope	`F08 · FieldRepository.relation()`	core

TEST FLOW



KNOWN LIMITATIONS OF THE TEST SCOPE

- ◆ Test coverage is focused on backend domain paths; frontend and browser paths are not covered.
- ◆ React Flow visualization (FieldGraph.tsx) is not covered by tests.
- ◆ One-command startup (start.sh; docker-compose.yml) is not end-to-end tested.
- ◆ No frontend test files exist.

◆ VERIFICATION BOUNDARY

Unverified execution paths: frontend build, Docker startup and browser E2E. These are reported as unverified, not as tested.

CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

06 VERIFICATION STATUS

EXECUTED · PENDING

◆ The recorded verification status per component. Left: what was executed. Right: what remains pending. Status follows TEST_REPORT.md and Volumes I and II.

COMPONENT	EXECUTED / RECORDED	PENDING VERIFICATION
Backend	5 / 5 backend tests passed (2026-11-01)	
Database (SQLite)	exercised through backend tests via FieldRepository	no migration framework; multi-connection operations not atomic
API	exercised via FastAPI TestClient in the 5 backend tests	no per-route test inventory documented
Frontend		build not executed; node_modules absent; no frontend tests
Docker		startup not executed; Docker unavailable
Browser E2E		not executed
Solution A	test_intent_trace passed	UI path not verified
Solution B	test_feedback_scope_and_confirm passed	UI path not verified
Solution C	test_gap_resolution passed	UI path not verified
Package	ZIP CRC integrity checked after packaging	

07 OPERATIONAL REFERENCE

RUNTIME · CONFIGURATION · SOURCE FILES · PROCEDURES

RUNTIME

ITEM	VALUE	SOURCE
Backend server	uvicorn at 127.0.0.1:8000	`start.sh`
Frontend server	Vite at 0.0.0.0:3000	`start.sh`
Browser	http://localhost:3000	`QUICK_START.md`
CORS	localhost:3000	`backend/api/main.py`
Database	ROOT/data/nemox.sqlite3 · CREATE TABLE IF NOT EXISTS at api.main import	`backend/fieldcore/core.py`
Vite proxy	local › 127.0.0.1:8000 · DOCKER_ENV=1 › backend:8000	Vol. I §10
Containers	python:3.12-slim :8000 · node:22-alpine :3000 · ./data:/app/data	Vol. I §10

CONFIGURATION FILES

FILE	LINES	ROLE
`README.md`	47	documentation / config
`backend/requirements.txt`	5	Python dependencies
`backend/Dockerfile`	8	backend runtime image
`frontend/Dockerfile`	6	frontend runtime image
`docker-compose.yml`	10	two-service compose
`start.sh`	17	one-command local start
`frontend/package.json · tsconfig.json · vite.config.ts`	1 · 1 · 2	frontend build and proxy

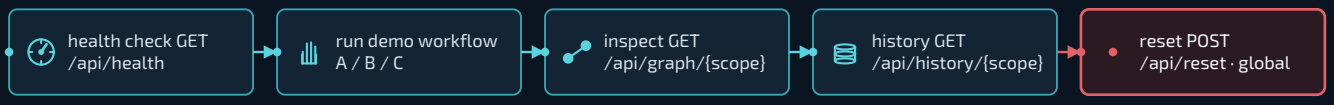
RELEVANT SOURCE FILES

FILE	OPERATIONAL ROLE
`backend/api/main.py`	routes, Pydantic models, ValueError › 400, init() at import
`backend/fieldcore/core.py`	FieldRepository, FieldContextResolver, state transitions, SQLite schema

CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

FILE	OPERATIONAL ROLE
backend/intent_inspector/service.py	Solution A · inspect()
backend/feedback_engine/service.py	Solution B · decide, correct, confirm, retrieve, subsequent, history
backend/gap_detector/service.py	Solution C · capability, analyze, evaluate, resolve
backend/tests/test_system.py	five backend tests
frontend/src/main.tsx · services/api.ts · components/FieldGraph.tsx	App state and tabs · API client · graph view

OPERATING PROCEDURES



OPERATING BOUNDARY

Reset is a global destructive demo reset across all scopes; there is no tenant authorization and no cross-tenant enforcement on reset.

CONFIDENTIAL · PROPRIETARY · NO USE, REPRODUCTION, DISCLOSURE OR IMPLEMENTATION WITHOUT PRIOR WRITTEN APPROVAL OF OTTAVIO BRAUN
© 2026 OTTAVIO BRAUN · SYNTX SYSTEM · MUNICH · ALL RIGHTS RESERVED

LEGAL NOTICE | CONFIDENTIALITY | RESERVATION OF RIGHTS

BINDING FOR EVERY RECIPIENT, READER OR HOLDER OF THIS DOCUMENT

1 Ownership

This document — including its structure, SYNTX representation, architecture reconstruction, recursive and inverse sweep results, diagrams, status classification and all derived descriptions — is the confidential intellectual work of Ottavio Braun, SYNTX SYSTEM. All rights are reserved.

2 No licence

No right of use of any kind is granted. Receipt, possession, access to or knowledge of this document does not constitute a licence, transfer of rights, offer, order, contract, partnership or commitment of any kind.

3 Written approval required

Any use, reproduction, storage beyond inspection, distribution, disclosure to third parties, adaptation, translation, implementation, re-engineering, integration into products or services, and any use for training, evaluating or prompting AI systems — in whole or in part, directly or indirectly — requires the prior, explicit, written approval of Ottavio Braun. Approval is valid only if confirmed by Ottavio Braun personally, in text form, for a named purpose and recipient. Silence, conduct, payment or prior collaboration do not constitute approval.

4 Purpose limitation

Until such approval is given, this document may be read solely to evaluate whether a written agreement with Ottavio Braun is to be concluded. Any other purpose is excluded.

5 Return and deletion

On request, and in any case if no written approval is granted, all copies — physical and digital, including excerpts, notes and derived material — must be deleted or returned without delay and their deletion confirmed in writing.

6 No warranty

The document describes an inspected source-code state as of 1 November 2026. It makes no representation of completeness, correctness, fitness for a particular purpose, security or production readiness. Implemented, derived, conceptual and proposed elements are distinguished; none of them is warranted.

7 No liability

No liability is accepted for decisions, implementations or damages arising from use of this document without the written approval described in section 3.

8 Third-party names

nemox, FastAPI, React, Vite, SQLite, Docker, React Flow / @xyflow and other product names remain the property of their respective owners and are used solely to describe the inspected implementation.

9 Proprietary method

The SYNTX Recursive Deep Sweep and Inverse Sweep are proprietary procedures of Ottavio Braun. Their internal material is not reproduced and no right in them is conveyed by this document.

Deutsch · verbindlicher Hinweis

Vertraulich. Jede Nutzung, Vervielfältigung, Weitergabe, Bearbeitung oder Umsetzung dieses Dokuments — ganz oder teilweise — bedarf der vorherigen, ausdrücklichen, schriftlichen Freigabe durch Ottavio Braun. Ohne diese Freigabe wird kein Nutzungsrecht eingeräumt.

APPROVAL CONTACT

Ottavio Braun | SYNTX SYSTEM

mirror@syntx-system.com

MUNICH, NOVEMBER 2026

© 2026 OTTAVIO BRAUN · ALL RIGHTS RESERVED